Katedra Grafiki Wizji Komputerowej

i Systemów Cyfrowych

Raport z projektu SMiW

Politechnika Śląska

Rok akademicki	Rodzaj studiów*: SSI Glw/SSI Ktw/NSI Glw	Dizeń tygodnia i godzina zajęć:	Numer sekcji:
2025/2026	**SSI Glw**	**Środa 11:30**	**7**
Imię:	**Mateusz**	**Prowadzący** (inicjały): OA/JP/GB/KP	**JP**
Nazwisko:	**Kowalec**		

Temat projektu

Smart Calendar

Opis głównych założeń projektu:

- **Wyświetlacz e-papier**
- **Pobiera dane o prognozie pogody oraz wydarzenia z kalendarza użytkownika z Internetu**
- **Wyświetla aktualną godzinę oraz zmierzoną temperaturę i wilgotność otoczenia**
- **Konfiguracja ustawień sieci przez BT**

Rodzaj zasilania:

- **Zasilanie bateryjne (AA NiMH)**

Szczegółowy opis funkcjonalności projektu:

Celem projektu jest stworzenie „mądrego” kalendarza. Podobnie do wersji papierowej, będzie można zaobserwować na nim aktualną datę, dowiedzieć się, czy danego dnia nie wypada święto, czyje są imieniny itd. Jedną z przewag takiego urządzenia nad tradycyjnym jest to, że po zamontowaniu w miejscu docelowym wymaga jedynie okazjonalnej wymiany baterii, a nie corocznej zmiany.

Jako urządzenie „mądre” będzie pokazywać prognozę pogody na najbliższe parę tygodni, pobraną z ogólnodostępnego API, Open-Meteo, oraz wydarzeń z kalendarza użytkownika w Google Calendar. Możliwe jest „przybliżenie” konkretnego dnia, co pozwoli wyświetlić dokładniejszą prognozę pogody (godzinową) oraz wydarzenia występujące w danym dniu. Dla dodatkowej wygody kalendarz będzie również wyświetlać aktualną godzinę, temperaturę, oraz wilgotność – jednocześnie zastępuje więc kalendarz, planer, zegar, pogodynkę, termometr oraz higrometr.

By zwiększyć komfort użycia (i zmniejszyć zużycie baterii) wykorzystany został wyświetlacz e-papier. Celem wydłużenia żywotności baterii, urządzenie będzie spędzać większość czasu w stanie spoczynku, wybudzając się jedynie co minutę w celu aktualizacji czasu na wyświetlaczu. Natomiast raz na konfigurowalny okres czasu urządzenie będzie wybudzać się celem połączenia z internetem, pobraniem nowych danych pogodowych oraz kalendarzowych, zsynchronizowanie czasu, wyłączenia radia, zaktualizowania wyświetlacza oraz jak najszybszego odcięcia sobie zasilania.

Interakcja użytkownika, rozpoczynana przyciskiem włączenia na płytce, wykonywana przełącznikiem nawigacyjnym podobnym w obsłudze do selektora na pilocie do telewizora, wybudza urządzenie z uśpienia, do którego powróci po dłuższym czasie bezczynności.

By nie musieć przeprogramowywać urządzenia w przypadku zmian ustawień sieciowych lub zmiany lokalizacji, dane o WiFi będą aktualizowane za pomocą aplikacji na telefonie (ESP WiFi provisioning, oryginalnie miało być to BLE, ostatecznie jest SoftAP, więcej we wnioskach).

Udzielanie dostępu do usług Google użytkownika jest wykonywane za pomocą przesłania kodu dostepu Google API przy pierwszym uruchomieniu urządzenia przez kabel USB podpięty do komputera. Urządzenie będzie zasilane bateriami AA NiMH. Bateriami, bo domyślnie ma zostać zamontowane na ścianie lub przypięte do tablicy korkowej, gdzie będzie wisieć nieściągane przez cały swój okres używania, a akumulatorki NiMH – bo takie wraz z ładowarką mam pod ręką.

Wybory najważniejszych części:

Czujnik wilgoci/temperatury:

	HDC3020	HDC1000YPAR	Sensirion SHTC3	Sensiron SHT40
Dokładność pomiarów	+/- 0,1C, +-0,5%	+/- 0,2C, +-3%	+-0,2C, +-2%	+-0,1C +-1.0%
Zakres temperatur	-40 do 125C	-40C do 125C	-40C do 125C	-40 do 125C
Interfejs	I2C	I2C	I2C	I2C
Napięcie zasilania	1.6V-5.5V	3V - 5V	1,62V-3,6V	1,08V-3,6V
Czas reakcji pomiaru RH	4s	15s	8s	6s

Sprawiłby się tu dowolny czujnik o dokładności temperatury ok. +/- 0.1C i wilgotności +/- 1%. Oryginalnie planowałem zamówić sam czujnik HDC3020, natomiast okazał się on zbyt mały by ręcznie go lutować (szczególnie, że nie mam płyty grzewczej). Zamówiłem więc breakout board SHT40 Adafruita, bo również spełnia moje wymagania, był w momencie zamawiania części dostępny w polskich magazynach i jest w akceptowalnej dla mnie cenie.

Przełącznik 5-kierunkowy:

Oryginalnie planowałem wybrać Adafruit SKQUCAA010, natomiast w celu zmniejszenia kosztów dostawy zamówiłem od innego dostawcy prawie identyczny przełącznik E-Switch JS5208. Projekt działałby praktycznie identycznie gdyby zamiast tego przełącznika dać np. 5 tact switchy, natomiast pojedyńczy przełącznik wygodniej się obsługuje.

Przetwornica:

	Pololu D24V10F3	TI TPS62840	Analog Devices ADP2370
Maks. prąd wyj.	1A	750mA	3A
Min. napięcie wej.	3,5V	1,8V	3,2V
IQ	1-4mA	60nA	34uA

Sprawdziłby się tu dowolny moduł przetwornicy Buck 3.3V, umożliwiający pobór przynajmniej 1A przy VIn=4V (Dropout voltage @1A = 0.7V lub mniej).
Wybrałem Pololu D24V10F3, bo to gotowy i prosty w zastosowaniu moduł z induktorem na płytce, który na pewno przyda mi się do przyszłych projektów. Prąd spoczynkowy okazał się nie mieć tutaj znaczenia - w stanie czuwania układu wejście zasilania przetwornicy jest kompletnie odcięte.

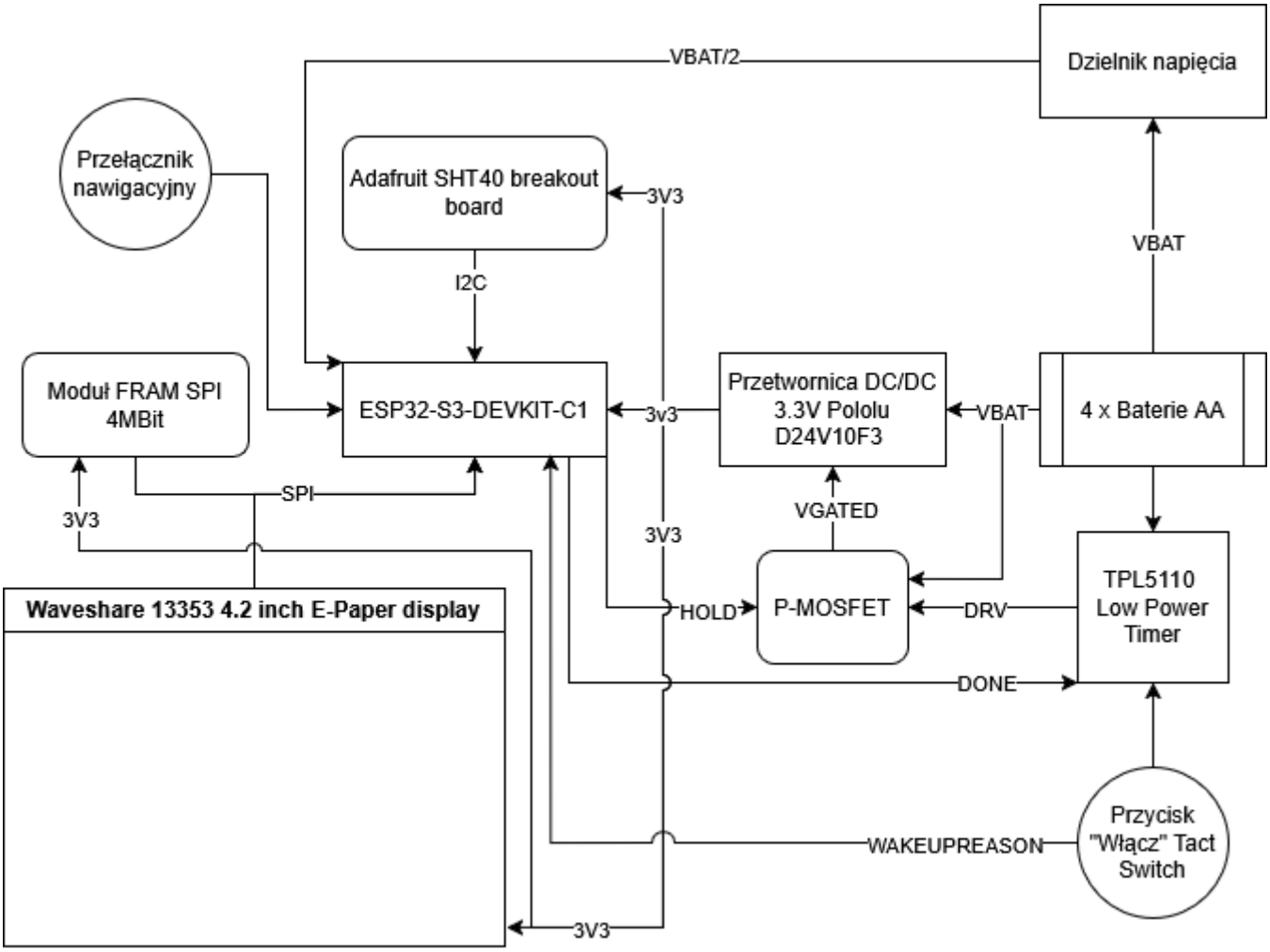
Mikrokontroler:

	ESP32-S3-DEVKIT-C1	ESP32-C6-DevkitM-1	ESP32-C3-DevKitM-1
Procesor	ESP32-S3-WROOM-1	ESP32-C3-MINI-1	ESP32-C6-MINI-1
Rdzenie	2	1	1
Pobór prądu w stanie uśpienia	Kilka uA		
GPIO	45	22	13
WiFi, BT	Tak	Tak	Tak

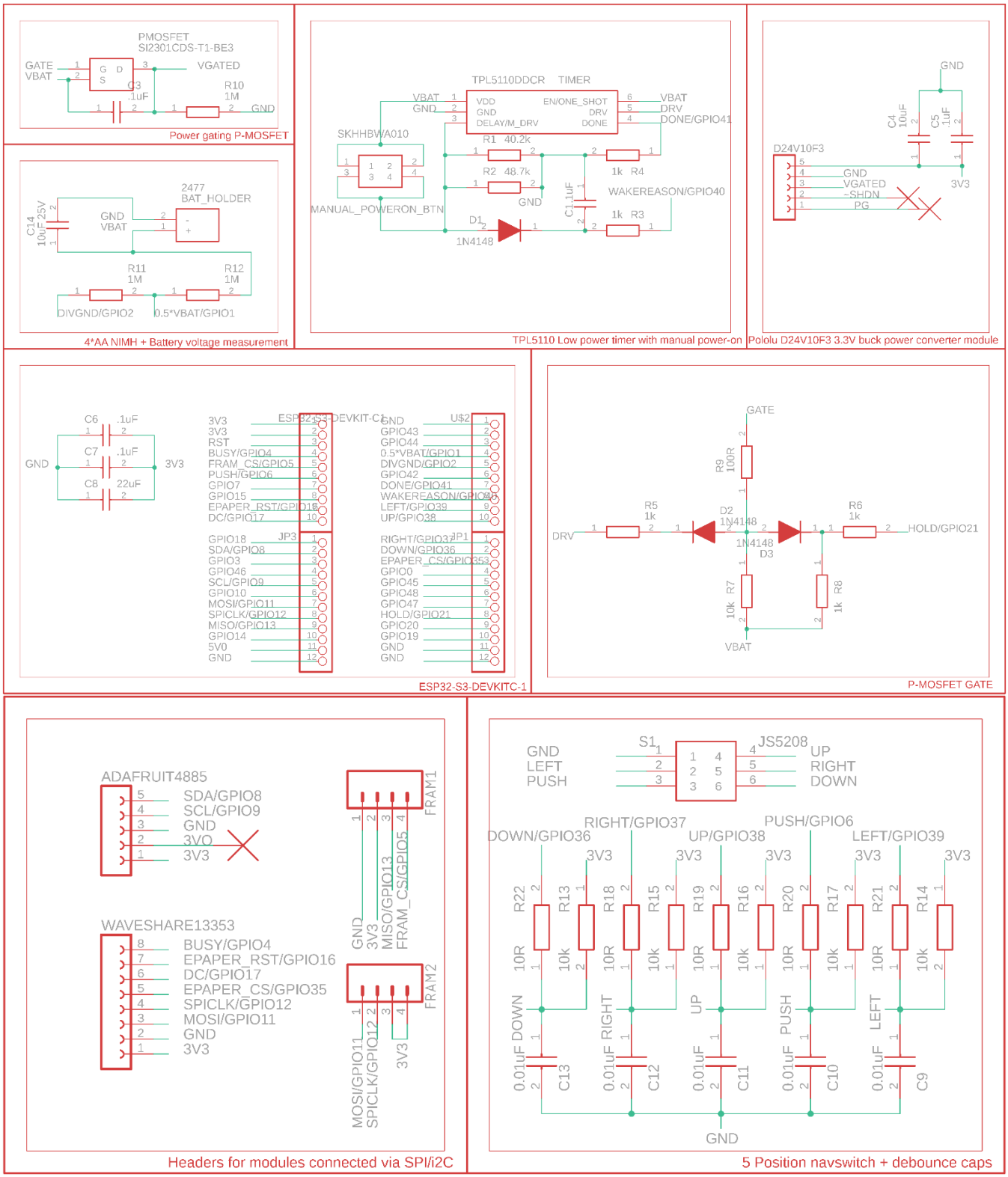
Wybrana została płytka rozwojowa ESP32-S3-DevKitC-1. Dużymi atutami tych mikrokontrolerów jest własna aplikacja do WiFi provisioning Espressif'a (ESP BLE/SoftAP Provisioning) + odpowiednia biblioteka, wbudowane wsparcie BT/WiFi + antena, niski pobór prądu (szczególnie w stanach uśpienia, choć końcowo nie miało to znaczenia), pełna kompatybilność z bibliotekami Arduino (pod warunkiem odpowiedniego skonfigurowania środowiska esp-idf) oraz niska cena. W ramach projektu z semestru letniego miałem już okazję zaznajomić się z procesorami z tej rodziny, a ten konkretny devkit już posiadam (nawet w dwóch egzemplarzach).

Projekt został zrealizowany w Autodesk EAGLE / ESP-IDF(Wersja linuksowa) + CLion + Arduino (IDF Component) .

Schemat blokowy urządzenia:



Schemat ideowy:



Opis funkcji bloków/elementów

Blok przetwornica – mikrokontroler – wyświetlacz + czujniki + przełącznik nawigacyjny raczej nie potrzebuje wytłumaczenia. O wiele ciekawszy natomiast jest blok bramkowania zasilania, składający się z TPL5110 Low Power Timer, tranzystora MOSFET z kanałem typu P oraz tact switcha.

Potrzebujemy wybudzać mikrokontroler tylko raz na minutę (celem aktualizacji informacji na wyświetlaczu: zegara oraz informacji z czujników – taki układ pozwala uzyskać prawie zerowy prąd spoczynkowy, wtedy kiedy tylko TPL5110 pobiera prąd max 50nA. Same działanie bloku jest proste: Tranzystor kontroluje przepływ prądu pomiędzy VBAT a VIN przetwornicy, odcinając przetwornice oraz całą resztę układu do niej podłączonego od zasilania.

TPL5110 co konfigurowalną (poprzez opór na wejściu DELAY/M_DRV) długość czasu, nazwijmy ją t, generuje stan niski na wyjściu DRV, podłączonym do bramki tranzystora. Bez żadnej interakcji ze strony mikrokontrolera układ utrzymywałby stan niski na DRV przez $t - 50ms$, po czym na ostatnie 50ms cyklu ustawiłby tam stan wysoki, odcinając na chwilę zasilanie i restartując układ. Mikrokontroler może przekazać timerowi informację o zakończeniu pracy, wysyłając odpowiedni impuls na wejście DONE, co skutkuje natychmiastowym odcięciem zasilania. Co ważne, timer generuje impulsy na DRV dokładnie co t, niezależnie od wysłania sygnału DONE.

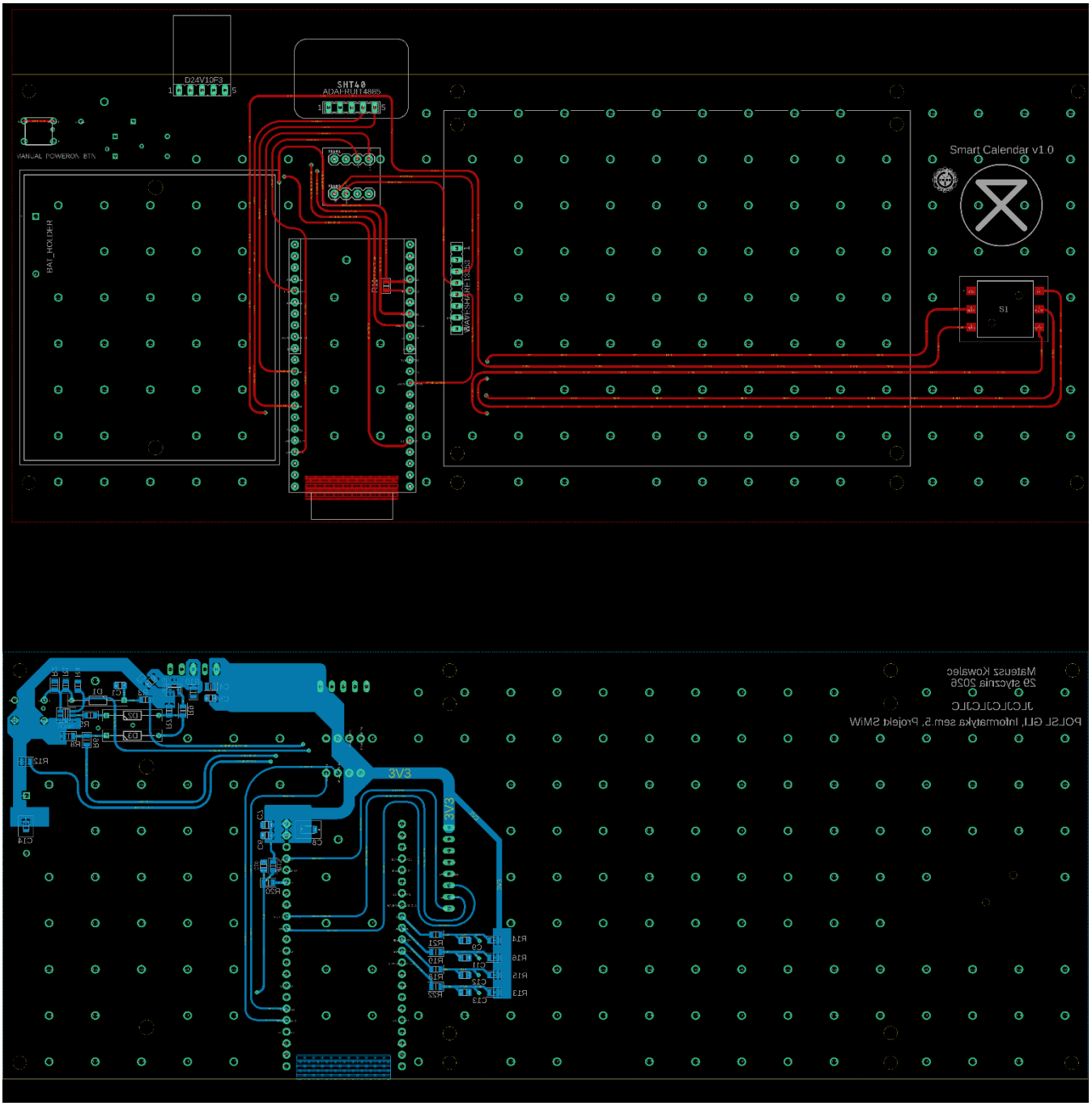
Timer ma również funkcję ręcznego włączenia, realizowaną poprzez osobny tact switch przy układzie zasilającym, podpięty do jego wejścia DELAY/M_DRV. Naciśnięcie przycisku skutkuje nadaniem napięcia VDD na to wejście, co powoduje ustawienie linii DRV na stan niski (nie wpływając na odliczanie czasu przez timer!). Przycisk jest również podłączony do procesora, wraz z kondensatorem "przetrzymującym" informację o naciśnięciu przycisku (timer reaguje na ręczne włączenie z lekkim opóźnieniem), co pozwala po włączeniu mikrokontrolera sprawdzić czy wybudzenie było automatyczne, czy ręczne, i odpowiednio dopasować tok programu. Oprócz timera, do bramki tranzystora podłączone jest również wyjście procesora, nazwane przeze mnie "HOLD", które procesor zaraz po wybudzeniu zwiera do masy. Dopuki HOLD = GND, procesor nie utraci zasilania, nawet jeśli timer odliczy minutę od ostatniego autmatycznego impulsu (DRV i HOLD przechodzą przez diody sygnałowe, mogą więc jedynie wpłynąć na bramkę ustawiając stan niski lub GND, co skutkuje włączeniem zasilania, natomiast żadne ze źródeł nie może wywołać tam stanu wysokiego, który jest domyślnie osiągany poprzez rezystor podciągający.

Cykl działania bloku wygląda więc następująco:

1. TPL5110 albo w skutek upłynięcia czasu t lub naciśnięcia przycisku ustawia DRV na stan niski
2. Mosfet przewodzi: Przetwornica zasila procesor
3. Procesor po włączeniu podtrzymuje zasilanie ustawiając hold na gnd
4. Procesor odczytuje informację o powodzie wybudzenia z linii WAKEREASON
5. Procesor wykonuje swoje zadania
6. Po zakończeniu działania, procesor najpierw wysyła impuls na linię DONE, by zasygnalizować TPL5110 by tamten przestał podtrzymywać zasilanie jeśli aktualnie to robi. Teraz DRV jest w stanie wysokim, i nie wpływa na bramkę tranzystora. Tylko HOLD utrzymuje teraz zasilanie.
7. Procesor ustawia HOLD na stan wysokiej impedancji, odcinając zasilanie do przetwornicy i co za tym idzie do reszty układu. Układ zużywa teraz znikomy prąd w oczekiwaniu na kolejne wydarzenie, które skutkowałoby wybudzeniem.

By jeszcze bardziej skapitalizować na bramkowaniu zasilania, do układu dodano moduł F-RAM 4Mbit RAMXEED (MB85RS4MTPF-G-BCERE1), co umożliwia cachowanie danych pobranych z internetu pomiędzy uruchomieniami. Pozwala to przez większość tych uruchomień nie włączać radia, a jedynie pobierać zcachowane dane z modułu. W NVS procesora przechowywany jest ostatni zsynchronizowany timestamp, oraz liczba wybudzeń do pełnej synchronizacji. Aktualny czas jest wyliczany ufając częstotliwości automatycznych wybudzeń jako $czas\ ostatniej\ synchronizacji\ +\ liczba\ wybudzeń\ od\ ostatniej\ synchronizacji\ *\ 60s$. Procesor tylko co parę automatycznych uruchomień / przy ręcznym pobiera świeże dane z internetu.

Projekt płytki/schemat montażowy



Zdjęcia warstwy górnej/dolnej płytki pcb. Pola masy, dla lepszej widoczności ścieżek i komponentów, zostały ukryte – w rzeczywistości wypełniają całe pozostałe wolne miejsce na płytce.

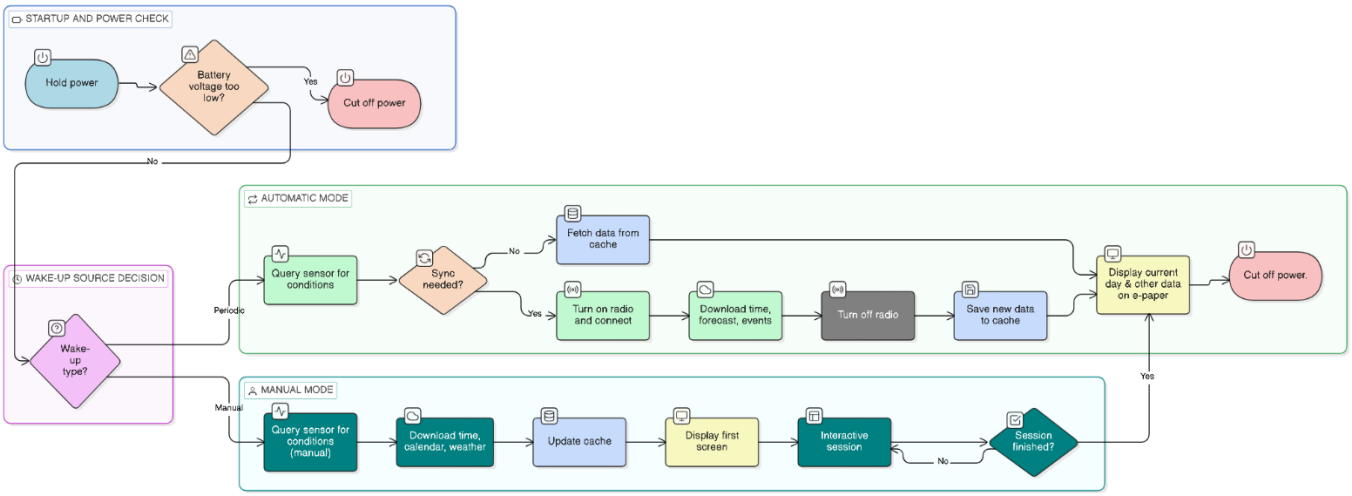
Lista elementów (BoM, export z EAGLE):

Part	Value	Device	Package	Description
Adafruit SHT40		ADAFRUIT4885		Temperature & Humidity Sensor
BAT_HOLDER	2477	2477	2477	Cylindrical Battery Contacts, Clips, Holders & Springs 4AA PLASTIC HOLDER
C1	.1uF	0805ZC104JAZ2A	CAPC2012X94N	Multilayer Ceramic Capacitors MLCC - SMD/SMT 10V .1uF X7R 0805 5% Tol FLEX
C3	.1uF	0805ZC104JAZ2A	CAPC2012X94N	Multilayer Ceramic Capacitors MLCC - SMD/SMT 10V .1uF X7R 0805 5% Tol FLEX
C4	10uF	GCM21BR71A106KE22L	CAPC2012X140N	Murata GCM21BR71A106KE22L 10uF Multilayer Ceramic Capacitor MLCC 10V dc +/-10% Tolerance SMD
C5	.1uF	0805ZC104JAZ2A	CAPC2012X94N	Multilayer Ceramic Capacitors MLCC - SMD/SMT 10V .1uF X7R 0805 5% Tol FLEX
C6	.1uF	0805ZC104JAZ2A	CAPC2012X94N	Multilayer Ceramic Capacitors MLCC - SMD/SMT 10V .1uF X7R 0805 5% Tol FLEX
C7	.1uF	0805ZC104JAZ2A	CAPC2012X94N	Multilayer Ceramic Capacitors MLCC - SMD/SMT 10V .1uF X7R 0805 5% Tol FLEX
C8	22uF	GRM31CR71A226KE15L	GRM31X	MLCC, 1206
C9	0.01uF	0805ZC103KAT2A	CAPC2012X94N	Capacitor 0805 10000PF +/-10% 10V
C10	0.01uF	0805ZC103KAT2A	CAPC2012X94N	Capacitor 0805 10000PF +/-10% 10V
C11	0.01uF	0805ZC103KAT2A	CAPC2012X94N	Capacitor 0805 10000PF +/-10% 10V
C12	0.01uF	0805ZC103KAT2A	CAPC2012X94N	Capacitor 0805 10000PF +/-10% 10V
C13	0.01uF	0805ZC103KAT2A	CAPC2012X94N	Capacitor 0805 10000PF +/-10% 10V
C14	10uF 25V	GCM21BC71E106KE36K	GCM21	Chip Multilayer Ceramic Capacitors for Automotive

				(Powertrain/Safety), 1.6 x 0.8 (0603)
D1	1N4148	1N4148	DIOAD1150W50L390D190	Diodes - General Purpose, Power, Switching Small Signal Diode, DO-35, 100V, 0.2A, 175C
D2	1N4148	1N4148	DIOAD1150W50L390D190	Diodes - General Purpose, Power, Switching Small Signal Diode, DO-35, 100V, 0.2A, 175C
D3	1N4148	1N4148	DIOAD1150W50L390D190	Diodes - General Purpose, Power, Switching Small Signal Diode, DO-35, 100V, 0.2A, 175C
D24V10F3		Pololu D24V10F3		Pololu 3.3V, 1A Step-Down Voltage Regulator D24V10F3
ESP32-S3-DEVKIT-C1			ESP32-S3-DEVKIT-C1	WiFi Development Tools - 802.11 WiFi Development Tools - 802.11 ESP32-S3 general-purpose development board, embeds ESP32-S3-WROOM-1, 8 MB Quad flash and 8MB Octal PSRAM, with pin header
FRAM1		CONN_04PTH	1X04	Multi connection point. Often used as Generic Header-pin footprint for 0.1 inch spaced/style header connections
FRAM2		CONN_04PTH	1X04	Multi connection point. Often used as Generic Header-pin footprint for 0.1 inch spaced/style header connections
FRAM		MB85RS4MTPF-G-BCERE1		F-RAM F-RAM 4Mbit FeRAM with SPI interface - SOP8 (208mil) T&R
JP1		PINHD-1X12-CB	1X12-CB	PIN HEADER
JP3		PINHD-1X12-CB	1X12-CB	PIN HEADER
MANUAL_POWERON_BTN		SKHHBWA010	SKHHALA010	Tact switch,PCB,SPST,grey, 1.57N op/force Grey Stem Tactile Switch, SPST-NO 50 mA 7mm
PMOSFET		SI2301CDS-T1-BE3	SOT95P237X112-3N	MOSFET P-CHANNEL 20-V (D-S)
R1	40.2k	CR0805-FX-4022ELF	RESC2012X60N	Thick Film Resistors - SMD 40.2K 1%
R2	48.7k	CR0805-FX-4872ELF	RESC2012X60N	Thick Film Resistors - SMD 48.7K 1%
R3	1k	CRCW08051K00FKTC	RESC2012X50N	VISHAY - CRCW08051K00FKTC

				- RES, THICK FILM, 1K, 1%, 0.125W, 0805
R4	1k	CRCW08051K00FKTC	RESC2012X50N	VISHAY - CRCW08051K00FKTC - RES, THICK FILM, 1K, 1%, 0.125W, 0805
R5	1k	CRCW08051K00FKTC	RESC2012X50N	VISHAY - CRCW08051K00FKTC - RES, THICK FILM, 1K, 1%, 0.125W, 0805
R6	1k	CRCW08051K00FKTC	RESC2012X50N	VISHAY - CRCW08051K00FKTC - RES, THICK FILM, 1K, 1%, 0.125W, 0805
R7	10k	RK73H2ATTE1002F	RESC2012X60N	Thick Film Resistors SMD1/4watts10Kohms 1%
R8	1k	CRCW08051K00FKTC	RESC2012X50N	VISHAY - CRCW08051K00FKTC - RES, THICK FILM, 1K, 1%, 0.125W, 0805
R9	100R	CRCW0805100RFKEA	RESC2012X50N	Vishay CRCW Series Thick Film Surface Mount Resistor 0805 Case 100?? ??1% 0.125W ??100ppm/??C
R10	1M	MCU08050C1004FP500	RESC2012X62N	1 MOhms +/-1% 0.2W, 1/5W Chip Resistor 0805 (2012 Metric) Anti-Sulfur Thin Film
R11	1M	MCU08050C1004FP500	RESC2012X62N	1 MOhms +/-1% 0.2W, 1/5W Chip Resistor 0805 (2012 Metric) Anti-Sulfur Thin Film
R12	1M	MCU08050C1004FP500	RESC2012X62N	1 MOhms +/-1% 0.2W, 1/5W Chip Resistor 0805 (2012 Metric) Anti-Sulfur Thin Film
R13	10k	RK73H2ATTE1002F	RESC2012X60N	Thick Film Resistors SMD1/4watts10Kohms 1%
R14	10k	RK73H2ATTE1002F	RESC2012X60N	Thick Film Resistors SMD1/4watts10Kohms 1%
R15	10k	RK73H2ATTE1002F	RESC2012X60N	Thick Film Resistors SMD1/4watts10Kohms 1%
R16	10k	RK73H2ATTE1002F	RESC2012X60N	Thick Film Resistors SMD1/4watts10Kohms 1%
R17	10k	RK73H2ATTE1002F	RESC2012X60N	Thick Film Resistors SMD1/4watts10Kohms 1%
R18	10R	CRCW0805100RFKEA	RESC2012X50N	Vishay CRCW Series Thick Film Surface Mount Resistor 0805 Case 100?? ??1% 0.125W ??100ppm/??C
R19	10R	CRCW0805100RFKEA	RESC2012X50N	Vishay CRCW Series Thick Film Surface

				Mount Resistor 0805 Case 100?? ??1% 0.125W ??100ppm/??C
R20	10R	CRCW0805100RFKEA	RESC2012X50N	Vishay CRCW Series Thick Film Surface Mount Resistor 0805 Case 100?? ??1% 0.125W ??100ppm/??C
R21	10R	CRCW0805100RFKEA	RESC2012X50N	Vishay CRCW Series Thick Film Surface Mount Resistor 0805 Case 100?? ??1% 0.125W ??100ppm/??C
R22	10R	CRCW0805100RFKEA	RESC2012X50N	Vishay CRCW Series Thick Film Surface Mount Resistor 0805 Case 100?? ??1% 0.125W ??100ppm/??C
S1		JS5208		JS5208 NAVIGATION, Surface Mount
TIMER		TPL5110DDCR	SOT95P280X110-6N	Timers & Support Products Ultr LowPowrTimerMOS drivr MOSFET Powr
U$2		PINHD-1X10	1X10-BIG	Pin header 1x10 0.1" spacing
WAVESHARE13353		WAVESHARE13353		4.2inch e-Paper Module



Algorytm oprogramowania

- Tok programu:
  - Podtrzymanie zasilania (HOLD = GND)
  - Sprawdzenie czy napięcie na baterii nie jest zbyt niskie (jeśli tak – od razu odcinamy zasilanie DONE = 1)
  - Sprawdzenie czy wybudzenie jest ręczne, czy okresowe
  - Jeśli okresowe:
    - Odpytać sensor o aktualne warunki.
    - Sprawdzić, czy od ostatniej synchronizacji nie minęło zbyt wiele uruchomień (licznik w nvs)
    - Jeśli jeszcze nie potrzebujemy synchronizacji:
      - Pobrać z fram-u ostatnio scachowane dane, z nvs pobrać poprzednio zapisany czas, zinkrementować go o jedną minutę, zapisać.
    - Jeśli jej potrzebujemy:
      - Włączyć radio, połączyć się z internetem.
      - Pobrać z internetu aktualny czas, prognozę, wydarzenia.
      - Wyłączyć radio.
      - Zapisać nowe dane w framie.
    - Wyświetlić zebrane dane na wyświetlaczu e-papier.
    - Odciąć zasilanie (HOLD = High-Z, DONE = 1).
  - Jeśli ręczne:
    - Odpytać sensor o aktualne warunki.
    - Pobrać z Internetu aktualny czas, dane z kalendarza i o prognozie pogody.
    - Zaktualizować cache w framie i dane w nvs.
    - Wyświetlić pierwszy ekran na wyświetlaczu.
    - Od tej pory zaczyna się sesja interaktywna – użytkownik porusza się po GUI, przeglądając daty lub zmieniając ustawienia.
    - Na życzenie użytkownika lub po minięciu jakiegoś czasu bez interakcji przechodzimy do obsługi jak w okresowym wybudzeniu.

Zmienne i procedury

Podczas pisania programu korzystałem ze strategii „dziel i zwyciężaj”, rozkładając wszystkie pożądane funkcjonalności na elementarne funkcje, o coraz bardziej specyficznym działaniu, odpowiednio sklejane by uzyskać pożądany efekt. Skutkuje to zbyt dużą liczbą małych funkcji, by opisać wszystkie. Opiszę więc tylko te najważniejsze:

- void staticDraw(const Date &d, tm now)
  - Wyświetla na wyświetlaczu epapier informacje pobrane wcześniej z sensorów oraz dane z cache’a na temat konkretnej daty. Wykorzystywany przy automatycznym wybudzeniu lub przy wyjściu z ręcznego by pokazać podgląd aktualnego dnia.
- void loopUI(Date currentDate)
  - Inicjalizuje pętle trybu interaktywnego, zaczynając od podanej daty.
- void manual() i void automatic()
  - Główne funkcje rozdzielające tok program dla konkretnego sposobu wybudzenia.
- void end()
  - Odcięcie zasilania (DONE = 1, HOLD = HIGH-Z)
- std::multimap<Date, Event> event_map;
  - Po pobraniu danych (z internetu lub cache) grupuje wydarzenia oraz dzień w którym się odbywają.
  - Zmienna globalna, inicjalizowana tylko raz.
- std::map<Date, Forecast> forecast_map;
  - Po pobraniu danych (z internetu lub cache) przechowuje prognozę pogody dla dni dla których ją mamy.
  - Zmienna globalna, inicjalizowana tylko raz.
- void networkSync()
  - Wewnętrznie składa się z wielu funkcji, np. connect(), getEvents(), loadEvents() itp.
  - Synchronizuje czas, wydarzenia oraz prognozę pogody z internetu, wypełniając wymienione wyżej zmienne oraz zapisując efekt do cache.
- void loadFromCache() i void saveToCache()
  - Zapisują lub pobierają dane z/do framu.
- String getAccessToken()
  - Zwraca access token jeśli mamy takowy w NVS, jeśli nie – generuje nowy za pomocą refresh tokena, jeśli go mamy. Jeśli nie mamy refresh tokena – zwraca pusty String.
- tm getTime()
  - Sprawdza czy zsynchronizowana już czas. Jeśli tak – zwraca go. Jeśli nie – synchronizuje go z internetem i zwraca uzyskany czas.
- float battery_voltage()
  - Mierzy i zwraca napięcie baterii w woltach.

Ze względu na liczbę funkcji zostały one w większości pogrupowane do wielu plików nagłówkowych, o nazwie takiej, jak ich zastosowanie.

Specyfikacja zewnętrzna

Sterowanie urządzeniem odbywa się za pomocą jedynie dwóch elementów kontrolnych: przycisku włączenia, w lewym górnym rogu urządzenia, oraz przełącznika nawigacyjnego, po prawej stronie wyświetlacza. W stanie domyślnym wyświetlacz wyświetla dane z poprzedniego uruchomienia, które miało miejsce nie więcej niż minutę temu: czas, dane z sensorów, napięcie baterii oraz wydarzenia/prognoza na aktualną datę, jeśli takowe istnieją. Konkretniej: Na pasku górnym wyświetla się aktualny czas, temperatura, wilgotność, SSID stacji Wi-Fi oraz napięcie na baterii; W dolnej części zależnie od trybu albo pojawia się aktualna data wraz z ogólną prognozą + tabelą prognozy godzinowej i liczbą wydarzeń w danym dniu lub w trybie wyświetlania wydarzeń – konkretne wydarzenie (tytuł, opis, godzina początku/końca).

Przycisk włącz służy jedynie do wejścia w tryb interaktywny. W trybie tym przełącznik nawigacyjny pozwala, za pomocą ruchu lewo-prawo, zmieniać datę dla której wyświetlamy dane. Dla dat które mają wydarzenia, możemy nacisnąć przycisk (PUSH), by wejść w tryb przeglądania wydarzeń – wtedy ruch lewo/prawo zmienia wydarzenie, występujące tego dnia, które przeglądamy. Ponowne naciśnięcie przycisku (PUSH) wraca do przeglądania prognoz/dat. W dowolnym momencie ruch góra/dół zaznacza lub oznacza pasek górny. Kiedy ten jest zaznaczony, można lewo/prawo wybrać tryb Provisioning albo Token refresh. Potwierdamy wybór naciśnięciem przycisku lub wychodzimy z paska górnego ruszając w dół. W trybie Provisioning korzystamy z instrukcji w aplikacji ESP SoftAP Provisioning, i czekamy aż układ się zsynchronizuje. W trybie Token refresh podpinamy urządzenia do komputera, i za pomocą np. idf.py monitor przesyłamy (wklejamy) access code do API Google, z minimum uprawnieniem Calendar.ReadOnly.
Wyjście z trybu interaktywnego jest albo automatyczne po 30s nieaktywności użytkownika lub po przytrzymaniu „dół” przez 3s.

Montaż, programowanie, uruchamianie

Montaż układu najlepiej zacząć od przylutowania komponentów SMD na dolnej stronie płytki, następnie na górnej. Szczególną uwagę należy zachować przy lutowaniu układu TPL5110, gdyż ma on bardzo mały odstęp pomiędzy pinami – trzeba dopilnować, by nie utworzyły się mostki. Następnie należy przylutować headery na wyświetlacz oraz mikrokontroler (najlepiej je wpiąć podczas lutowania by nie wyszło krzywo). Wyświetlacz wypada dodatkowo przykręcić – wszystkie otwory na płytce są dopasowane do śrubek M3, dobrze zaopatrzyć się również w odpowiednie dystanse oraz nakrętki, najlepiej wszystko plastikowe. Moduł FRAM jest zamontowany na przejściówce SOP-8 -> DIP 8-pin – został do niej przylutowany by móc testować go na płytce stykowej. Uznałem, że nie chcę ryzykować z wylutowywaniem go z tej przejściówki, gdyż mam tylko jeden, a moduły FRAM są dość drogie. Moduł fram, przetwornice oraz sensor sht40 polecam zamontować poprzez bezpośrednie przylutowanie męskich headerów do płytki, pomijając żeńskie, by obniżyć profil urządzenia. Koszyk na baterie, który okazał się większy o ok. 1cm od danych producenta, najlepiej zamontować na sam koniec – ostatecznie jest bardzo mało miejsca między jego krawędzią a krawędziami headerów płytki procesora.

Cały układ został polutowany oraz złożony za pomocą standardowej lutownicy oraz cyny z dodatkiem ołowiu, na płytce zamówionej na JLCPCB.

Dla celów testowych użyty przeze mnie devkit miał już wylutowany stabilizator napięciowy (by móc debugować go nie wpływając na jego zasilanie). Jeśli chce się użyć niezmodyfikowanej płytki, trzeba pamiętać by przy programowaniu/przesyłaniu klucza nie mieć włożonych baterii, by układ na pewno był zasilany tylko z USB, lub po prostu wyciągnąć procesor z płytki.

Układ podpinamy do komputera, kompilujemy kod (najlepiej z folderu projektu (po wykonaniu `source export.sh` w folderze esp-idf oraz `idf.py set-target esp32s3` w folderze projektu): `idf.py –ccache flash`. Dla zmodyfikowanej płytki przy debugowaniu wykonywałem `idf.py –ccache flash size monitor`.

Po zaprogramowaniu układu należy w opisany wcześniej sposób przekazać mu dane o Wi-Fi oraz kod do API Google. Układ nie potrzebuje dalszej konfiguracji.

Oprócz ręcznego testowania każdej funkcji, układ został również przetestowany poprzez zwykłe go użytkowanie – dodawanie wydarzeń do kalendarza Google i sprawdzanie czy wyświetlają się na nim, sprawdzanie prognozy pogody, zostawianie go włączonego na długi okres czasu, zmiana paremetrów stacji Wi-Fi routera itd.

Wnioski, uwagi, problemy

Przede wszystkim: cały tok pracy z ESP32S3 okazał się bardzo przyjemny. IDF (przynajmniej na linuksie) szybko buduje projekt, jest w pełni zintegrowany z CLionem (szczególnie debugger świetnie działa). Dodawanie bibliotek Arduino jest również proste i wymaga w najgorszym przypadku dodania tylko pliku CMakeLists.txt ze stałą zawartością do folderu biblioteki. Możliwość zmiany ustawień interaktywnie przez menuconfig znacznie bardziej przypadła mi do gustu niż wpisywanie ich ręcznie w np. PlatformIO. Sam procesor ma naprawdę dużo funkcji, szczególnie za swoją cenę – więcej GPIO niż mam pomysłów na co je użyć, BLE + WiFi, kalibrowane pomiary napięcia, debugger, nvs, 16MB RAMu, dwa rdzenie na których łatwo mogę rozkładać pracę Taskami FreeRTOSa, pełna kompatybilność z bibliotekami Arduino wraz z możliwością korzystania/mieszania z bibliotekami esp-idf… Jestem bardzo zadowolony z produktu i zdecydowanie przy przyszłych projektach będę sięgać po procesory Espressifa.

Jeśli chodzi o problemy, wystąpiły zasadniczo dwa:

BLE Provisioning – oryginalnie, zgodnie z założeniami projektu, chciałem udostępniąć dane o Wi-Fi przez Bluetootha. Niestety, coś w aktualnej wersji arduino-esp32 uniemożliwiało mi skompilowanie `scheme_ble.c`, niezależnie od opcji w ustawieniach projektu CMake lub menuconfig. Po paru godzinach zabawy uznałem, że skoro pliki SoftAP kompilują się prawidłowo, a nie ma zbyt dużej różnicy pomiędzy ich używaniem (jeden parametr w funkcji provisioning ulega zmianie a użytkownik musi po prostu zainstalować inną aplikację), to po prostu zamienię sposób udostępniania połączenia.

Układ procesora w EAGLE – by uniknąć problemów z layoutem, procesor wraz z narysowanym przeze mnie silkscreenem oraz keepoutem pod anteną zostały przeze mnie pogrupowane do jednej grupy. W trakcie tworzenia projektu zachciało mi się obrócić procesor o 180 stopni, by antena wystawała poza płytkę, a wejścia do programowania dalej były łatwo dostępne (bo moduł FRAM jest niższy). Niestety, popełniłem na tym etapie jakiś błąd – którego nie zauważyłem przed potwierdzeniem produkcji płytki: Keepout wraz z rysunkiem modułu obróciłem, ale same headery pozostały w starej pozycji. Na szczęście, układ działa poprawnie, po prostu procesor jest „na odwrót” niż jak chciałem, a pod anteną na głównej płytce nie ma keepouta. Ze względu na podwyższenie procesora (jest w końcu wpięty header-header) względem płytki, nie powinien to być zbyt duży problem.

Poza tym – wszystko poszło gładko i przyjemnie, a układ działa według oczekiwań.